

**Aplikacja webowa
do zgodnej z obostrzeniami sanitarnymi
rezerwacji miejsc w pomieszczeniach**

(Web application for reservation workstations in offices
according to sanitary restrictions)

Ireneusz Gościński

Praca inżynierska

Promotor: dr inż. Leszek Grocholski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

3 lutego 2022

Streszczenie

Pandemia koronawirusa wpłynęła na wiele obszarów życia codziennego, najczęściej mocno je utrudniając. Jednym ze skutków jest konieczność pracy zdalnej lub reorganizacja pracy stacjonarnej w taki sposób, aby zminimalizować ryzyko zakażenia. Pracodawcy są zobligowani do przeprowadzenia tego zadania zgodnie z panującymi obostrzeniami sanitarnymi. Określają one najczęściej maksymalną procentową zajętość miejsc oraz minimalną odległość między stanowiskami pracy. Celem tej pracy było napisanie aplikacji internetowej, która ułatwiłaby firmom czy instytucjom publicznym organizację pracy zgodnie ze wspomnianymi wytycznymi. Zostało to zrealizowane przez system umożliwiający pracownikom firmy rezerwację miejsc jedynie z określonej puli, dobranej w taki sposób, aby obostrzenia były zachowane. Parametry wpływające na wyliczanie zbioru dostępnych miejsc mogą być modyfikowane, co umożliwia korzystanie z aplikacji przy zmieniających się restrykcjach, dostosowywanych do sytuacji epidemicznej.

The coronavirus pandemic has affected many areas of daily life, most often making it more difficult. One consequence is the requirement to work remotely or to reorganize office work in such a way as to minimize the risk of infection. Employers are obliged to conduct this task in accordance with current sanitary regulations. These usually specify a maximum percentage occupancy and a minimum distance between workstations. The aim of this project was to design a web application that would make it easier for companies or public institutions to organise work according to these restrictions. This was achieved by a system that allows company employees to reserve places only from a specific set of seats, chosen in such a way that the regulations are respected. The parameters that influence the calculation of the pool of available places can be modified, allowing the application to be used with changing restrictions according to the epidemic situation.

Spis treści

1. Wprowadzenie	7
1.1. Wstęp	7
1.2. Cel i zakres pracy	7
1.3. Opis zagadnienia	8
1.4. Inne rozwiązania	8
2. Część dla użytkowników	11
2.1. Ogólny opis programu	11
2.2. Sposób dostępu do systemu	12
2.3. Podręcznik użytkownika	12
3. Część dla programistów	17
3.1. Narzędzia użyte w projekcie	17
3.2. Schemat bazy danych	18
3.3. Opis struktury projektu	19
3.4. Testowanie	20
4. Zakończenie	23
4.1. Podsumowanie	23
4.2. Możliwe kierunki rozwoju	23
Bibliografia	25

Rozdział 1.

Wprowadzenie

1.1. Wstęp

Wśród wielu wyzwań, jakie postawiła przez ludzkością pandemia, pojawia się nowa organizacja pracy. Znaczna część firm, instytucji czy uczelni przeszła na tryb zdalny. Nie zawsze jest to jednak możliwe, zdaniem wielu taka forma jest mniej efektywna i powinna pozostać jedynie rozwiązaniem przejściowym. Chcąc zachować stacjonarny tryb pracy, musimy dostosować się do obostrzeń, ustalonych aby ograniczyć rozprzestrzenianie się wirusa. Zostały one wprowadzone przez Rozporządzenie Rady Ministrów z dnia 10 kwietnia 2020 roku[8, § 9, pkt. 9, ppkt. 3] i utrzymywane w mocy przez liczne, kolejne rozporządzenia, między innymi z 19 czerwca[9], czy 7 sierpnia[10]. To rodzi kolejne problemy dla pracodawców, np. jak zapewnić każdemu pracownikowi miejsce, jeśli co drugie musi pozostać puste? Z takimi trudnościami można radzić sobie bez korzystania z dodatkowego oprogramowania, np. poprzez przyklejanie kartek z informacją, gdzie można usiąść. Rozwiązanie takie nie umożliwia jednak wcześniejszej rezerwacji miejsca, co więcej jest też mało elastyczne — jeśli np. zmieni się ograniczenie dotyczące procentowej zajętości miejsc, jesteśmy zmuszeni wykonać tę samą pracę jeszcze raz. Takie problemy zawsze otwierają przestrzeń dla tworzenia nowego oprogramowania użytkowego, najczęściej w postaci aplikacji internetowych czy mobilnych.

1.2. Cel i zakres pracy

Odpowiedzią na problem rezerwacji miejsc jest niniejsza praca inżynierska. Jej celem jest zaprojektowanie i zaimplementowanie aplikacji, która ułatwiałaby zarówno pracodawcom jak i pracownikom organizację pracy stacjonarnej zgodnie z przepisami. Zdecydowałem się wykonać ją jako aplikację webową — taka forma nie wymaga żadnej instalacji i jest dobrze znana, co powinno znacząco ułatwić jej użytkowanie. Od strony problemu pracy, do jej zakresu należeć będą rezerwacja miejsc

zgodnie z obostrzeniami, przeglądanie historii oraz możliwość usuwania przyszłych rezerwacji. Ze względu na wybór formy realizacji aplikacji, musi ona umożliwiać zakładanie konta (w tym przypadku mamy dwa typy użytkowników), logowanie, zmianę danych użytkownika, w tym hasła.

1.3. Opis zagadnienia

Głównym zagadnieniem projektu jest tutaj sama rezerwacja miejsca, choć system umożliwia też ich przeglądanie czy anulowanie. Najważniejszym problemem przy rezerwacji, z punktu widzenia zgodności z obostrzeniami sanitarnymi, jest wybranie ze zbioru wszystkich stanowisk pracy w firmie takiego podzbioru, który będzie spełniał wspomniane restrykcje. Same ograniczenia mają dwa typy i do wyboru dostępnych miejsc można skorzystać z każdego z nich, jak również z obu naraz. Oprócz restrykcji miejsca mogą być już zajęte przez innych pracowników, co również należy uwzględnić konstruując podzbiór. Nieco osobnym zagadnieniem, ale również związanym z rezerwacją miejsca, jest poprawne obsłużenie sytuacji, gdy dwóch pracowników chce zarezerwować to samo miejsce jednocześnie.

Drugim aspektem projektu jest poprawne rozwiązanie zagadnień wynikających z wybranego typu aplikacji. Należy zadbać o prawidłową obsługę rejestracji i logowania, wraz ze wszystkimi kwestiami, jakie się z tym wiążą, jak np. walidacja danych czy bezpieczeństwo haseł. Kolejnym wyzwaniem jest poprawna obsługa żądań HTTP tak, aby użytkownik nie spotkał się z sytuacją, gdy serwer odpowiada komunikatem błędu. Należy wreszcie zwrócić uwagę na bezpieczeństwo — oprócz wspomnianego bezpieczeństwa haseł nie można dopuścić, aby użytkownik uzyskał dostęp do zasobów, do których nie ma uprawnień, np. niezalogowany użytkownik nie może mieć możliwości załadowania strony z historią rejestracji. Pomimo, że użyty framework upraszcza wiele problemów lub wręcz obsługuje je za programistę, nadal pozostaje wiele kwestii do zaprojektowania i zaimplementowania samodzielnie.

1.4. Inne rozwiązania

Z aplikacją rozwiązującą problem rezerwacji miejsc w biurze zgodnie z obostrzeniami sanitarnymi zetknąłem się podczas praktyk wakacyjnych w firmie Europejski Fundusz Leasingowy. Firma ta na początku pandemii zleciła Departamentowi Informatyki przygotowanie takiej aplikacji na swój wewnętrzny użytek. Wiązało się to z pewnymi rozwiązaniami projektowymi, które odróżniają tę aplikację od naszej pracy. Jest ona ograniczona tylko do jednej firmy, podczas gdy w naszym projekcie firmy czy instytucje mają możliwość zakładania konta, z którym następnie łączą się konta ich pracowników. Różne jest również podejście do umieszczenia w programie konkretnych restrykcji — w aplikacji EFL zdecydowano się ustawić liczbę dostępnych miejsc na 50%, bez możliwości modyfikacji. W przedstawionej pracy dostępne

miejsca są wyliczane na podstawie dwóch parametrów: maksymalnej dopuszczanej zajętości miejsc wyrażonej procentowo, oraz minimalnej odległości między sąsiednimi stanowiskami pracy. Oba te parametry mogą być modyfikowane przez administratora systemu, dzięki czemu aplikacja jest bardziej elastyczna pod względem dostosowywania się do zmieniających się obostrzeń.

Proces rezerwacji i ogólnie interfejs użytkownika co do funkcjonalności wyglądają podobnie — pracownik firmy może podejrzeć historię swoich rezerwacji, anulować te z nich, które dotyczą przyszłych terminów, oraz, przede wszystkim, dokonać rezerwacji. W tym miejscu aplikacje różnią się tylko jeśli chodzi o graficzny sposób przedstawienia dostępnych miejsc. W mojej pracy zdecydowałem się na wypisanie tylko dostępnych miejsc dla danego pracownika, podczas gdy aplikacja firmy EFL pokazywała również, kto zarezerwował inne miejsca w pobliżu.

Rozdział 2.

Część dla użytkowników

2.1. Ogólny opis programu

Aplikacja obsługuje dwa rodzaje użytkowników: firmę i pracownika. Istotnymi wartościami, wprowadzanymi podczas zakładania konta firmowego, są informacje dotyczące liczby stanowisk oraz odległości między nimi. Na podstawie tych parametrów wylicza się później zbiór dostępnych miejsc. Z kolei zakładając konto pracownika należy wybrać firmę, w której pracuje. Wszystkie wprowadzone dane można później zmodyfikować.

Pracownicy mają możliwość rezerwacji miejsc w firmie, którą podali przy zakładaniu konta. Po wyborze dnia wyświetlać się będą tylko dostępne miejsca, tak więc rezerwacje użytkownika nie są widoczne dla pozostałych. Pracownik może podejrzeć historię swoich rezerwacji, w szczególności tych dotyczących przyszłych terminów. Takie rezerwacje mogą być przez niego anulowane. Firma również ma możliwość podglądu historii rezerwacji oraz osobno rezerwacji na przyszłe dni, aby móc na bieżąco monitorować obłożenie miejsc.

Specyfikację programu można ubrać w następujące wymagania:

- Wymagania funkcjonalne:
 - obsługa dwóch typów użytkowników,
 - zakładanie konta,
 - możliwość zmiany danych użytkownika,
 - możliwość zmiany miejsca zatrudnienia przez pracownika,
 - tworzenie rezerwacji,
 - podgląd historii rezerwacji,
 - możliwość anulowania przyszłych rezerwacji (dotyczy pracowników),
 - możliwość zmiany parametrów wyliczania dostępnych miejsc

- Wymagania niefunkcjonalne:
 - intuicyjny interfejs,
 - możliwość elastycznego dopasowywania parametrów programu do zmieniających się restrykcji,
 - łatwa obsługa systemu,
 - niezależność od systemu operacyjnego i oprogramowania użytkownika

2.2. Sposób dostępu do systemu

Projekt jest aplikacją internetową, nie wymaga zatem żadnej instalacji. Jedynym oprogramowaniem, jakiego potrzebujemy, to przeglądarka internetowa. Aplikacja jest hostowana na platformie heroku.com, dzięki czemu jest do niej dostęp z każdego urządzenia podłączonego do sieci. Można ją znaleźć pod adresem <https://yourchair.herokuapp.com/>. Aby uruchomić aplikację bez łączenia się z internetem, mając dostęp do źródeł, należy uruchomić terminal w katalogu głównym projektu i wykonać polecenie

```
$ python manage.py runserver
```

Aplikacja będzie wtedy dostępna na lokalnym serwerze komputera na porcie 8000, tj. pod adresem `localhost:8000`. Jeśli chcemy użyć innego portu, wystarczy podać go jako parametr na końcu polecenia.

2.3. Podręcznik użytkownika

Rejestracja

Przechodzimy do podstrony `/register/` klikając przycisk „Sign Up” na górnym pasku, po prawej stronie. Wybieramy typ użytkownika, którego chcemy utworzyć: pracownika lub firmę. Naciskamy przycisk. W obu przypadkach podajemy *username* naszego użytkownika. Nazwa ta musi być unikatowa, gdyż będzie jednoznacznie wyznaczać użytkownika aplikacji. Wspólnym polem dla obu typów jest też *email*. Pracownik podaje swoje imię i nazwisko oraz wybiera z listy firmę, w której pracuje, natomiast firma podaje swoją nazwę oraz parametry do wyliczania miejsc: ich liczbę oraz odległość między nimi. Wartości te muszą być dodatnie. Konto obu typów użytkowników musi być zabezpieczone hasłem, które należy wprowadzić dwa razy. Program wymusza, aby hasła nie były zbyt proste, narzucając wymagania np. co do długości czy podobieństwa do innych danych użytkownika.

The screenshot shows a web browser window with a blue header bar. On the left of the header is the text 'YourChair'. On the right are the links 'Home', 'Log in', and 'Sign in'. Below the header is a 'Registration form'. The form contains the following fields and labels: 'Username*' with the value 'jan_kowalski' and a note 'Required, 150 characters or fewer. Letters, digits and @/+/./_ only.'; 'Email*' with the value 'kowalski@gmail.com'; 'First name*' with the value 'Jan'; 'Last name*' with the value 'Kowalski'; 'Password*' with a masked value '*****' and a list of password requirements: 'Your password can't be too similar to your other personal information.', 'Your password must contain at least 8 characters.', 'Your password can't be a commonly used password.', and 'Your password can't be entirely numeric.'; 'Password confirmation*' with a masked value '*****' and the instruction 'Enter the same password as before, for verification.'; and 'Company*' with the value 'firma2'. At the bottom of the form is a blue button labeled 'Sign up'.

Rysunek 2.1: Ekran rejestracji użytkownika typu pracownik

Logowanie

Znajduje się pod stroną `/login/`. Można do niej przejść przez przycisk „Sign In” na górnym pasku po prawej stronie. Strona logowania jest wspólna dla obu typów kont. Aby się zalogować, należy podać swoją nazwę użytkownika i hasło podane przy zakładaniu konta.

Profil

Do strony `/profile/` można przejść przez naciśnięcie przycisku na górnym pasku po prawej stronie (jest to możliwe tylko, gdy użytkownik jest zalogowany). Po przejściu do strony wyświetli się tabela z danymi użytkownika, wprowadzonymi podczas rezerwacji. Naciśnięcie przycisku „Update Data” spowoduje przejście do strony `/update-profile/`, gdzie możemy zmodyfikować dane. Można zaktualizować wszystkie dane, łącznie z nazwą użytkownika. W przypadku użytkownika będącego firmą zmiana parametrów branych pod uwagę przy wyliczaniu dostępnych do rezerwacji miejsc, tj. liczby stanowisk w firmie i odległości między nimi, spowoduje usunięcie wszystkich przyszłych rezerwacji, tak, aby nie doszło do niezgodności. Taki sam efekt spowoduje zmiana firmy przez pracownika, choć tutaj uzasadnienie jest inne — nie ma sensu, by pracownik zajmował miejsce w firmie, w której już nie pracuje. Z kolei naciśnięcie przycisku „Change password” spowoduje przejście do strony `/change-password/` i umożliwi zmianę hasła (należy standardowo dla takiej procedury podać obecne hasło i dwukrotnie nowe).

Rezerwacja miejsca

Ta funkcjonalność jest dostępna tylko dla pracowników. Do strony rezerwacji (`/make-a-reservation/`) można przejść przez wybranie „Make a reservation” z listy po lewej stronie. Pokazuje nam się strona do dokonywania rezerwacji. Pierwszą rzeczą,

Your profile

Field	Value
Username	user1
Email address	user1@gmail.com
First name	Jan
Surname	Kowalski
Company	Biuro rachunkowe

Update data

Change password

Updating profile information

Username*
jan_kowalski
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email*
kowalski@gmail.com

First name*
Jan

Last name*
Kowalski

Company*
firma2

Update

Rysunek 2.2: Profil użytkownika

Rysunek 2.3: Edycja profilu

którą musimy zrobić, jest wybór daty. Nie można wybrać daty z przeszłości (tj. starszej niż dzisiejsza) ani daty, kiedy mamy już jakąś rezerwację. Wybór zatwierdzamy przez naciśnięcie przycisku „Submit”. Nastąpi przeładowanie strony i jeśli nie popełniliśmy błędu wyświetlą nam się dostępne miejsca na dany dzień. Wyświetla się 8 miejsc na stronę, na dole znajdują się przyciski do przewijania stron. Gdy chcemy zarezerwować jakieś miejsce, wybieramy je przez naciśnięcie „Book” przy numerze stanowiska, które chcemy zarezerwować. Jeśli zdarzy się, że inny użytkownik chwilę przed nami zarezerwuje dokładnie to stanowisko, otrzymamy odpowiedni komunikat i będziemy musieli wybrać inne miejsce.

YourChair
Home Profil Log out

Menu
Make a reservation
Future reservations
History

Make a reservation
Select the date:
31/01/2022
Submit

Available work stations

Workstation number	
1	Book
4	Book
6	Book
9	Book
11	Book
14	Book
16	Book
19	Book

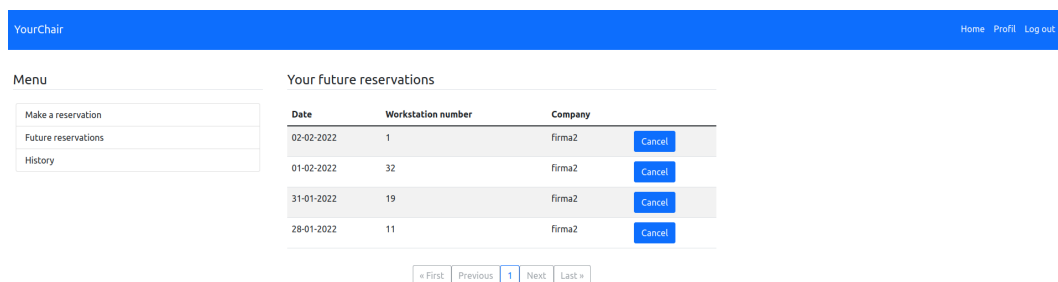
« First
Previous
1
Next
Last »

Rysunek 2.4: Rezerwacja miejsca

Przegląd dokonanych rezerwacji

Możemy przeglądać albo wszystkie dokonane rezerwacje, albo tylko te, które dotyczą przyszłych terminów. Do przejścia do widoków służą odpowiednio przyciski

„History” oraz „Future reservations” w menu po lewej stronie. Rezerwacje są wyświetlane po maksymalnie 14 na stronie. Na dole, poniżej listy, znajdują się przyciski do nawigacji między stronami. W listach z rezerwacjami dla pracowników znajduje się pole *Company*. Jest to spowodowane tym, że pracownik może zmienić firmę, w której pracuje. Pracownicy mają ponadto możliwość anulowania przyszłych rezerwacji. Aby to zrobić, należy przejść do strony z podglądem bieżących rezerwacji i nacisnąć przycisk „Cancel” przy tej z nich, którą chcemy anulować.



The screenshot displays the 'Your future reservations' section of the 'YourChair' application. On the left, a 'Menu' sidebar contains links for 'Make a reservation', 'Future reservations' (the active page), and 'History'. The main content area shows a table of reservations. The table has three columns: 'Date', 'Workstation number', and 'Company'. There are four rows of reservations, each with a 'Cancel' button. Below the table is a pagination control with buttons for 'First', 'Previous', '1' (the current page), 'Next', and 'Last'.

Date	Workstation number	Company	
02-02-2022	1	firma2	Cancel
01-02-2022	32	firma2	Cancel
31-01-2022	19	firma2	Cancel
28-01-2022	11	firma2	Cancel

« First Previous 1 Next Last »

Rysunek 2.5: Podgląd przyszłych rezerwacji pracownika

Rozdział 3.

Część dla programistów

3.1. Narzędzia użyte w projekcie

Python

Językiem programowania, w którym została napisana aplikacja, jest Python[1]. Język ten łączy prostotę składni i niski próg wejścia z dużymi możliwościami. Wybór ten był podyktowany szeroką gamą dostępnych frameworków i bibliotek, dedykowanych wszelkim dziedzinom programowania, posiadających wbudowane rozmaite rozwiązania odciążające programistę, dobrze udokumentowanych i na bieżąco uaktualnianych, co ma szczególną wagę w kwestiach związanych z bezpieczeństwem. Istotnie było też moje doświadczenie w pracy tym językiem, w szczególności jeśli chodzi o systemy obsługujące bazy danych.

Django

Frameworkiem przeznaczonym do tworzenia aplikacji internetowych, wybranym w mojej pracy, jest Django[2]. Jest to naczelny framework tego przeznaczenia dla języka Python[1], cieszący się dużą popularnością. Jego główne zalety to między innymi możliwość połączenia z różnymi bazami danych i to przy zachowaniu całkowitej przenośności kodu, generowany automatycznie panel administratora, czy rozdzielenie wyglądu stron od logiki ich działania i struktury bazy danych. Również automatycznie generowany szkielet projektu wspiera rozdzielenie go na poszczególne zadania, co zostanie opisane dokładniej w następnym podrozdziale. Wyczerpująca dokumentacja i zamieszczenie w niej przykładu zaprogramowania niewielkiej aplikacji czynią ten framework bardzo przyjaznym programiście. Django[2] posiada również wsparcie dla tworzenia testów, w tym testów automatycznych.

Crispy

Niewielką, ale bardzo użyteczną biblioteką języka Python[1], jest Django-crispy-forms[7]. Umożliwia wyświetlanie formularzy w przyjaznym, przejrzystym układzie,

oraz zapobiega powtarzaniu się kodu. Jest również niezwykle prosta w obsłudze — od strony logiki działania strony pozostaje całkowicie przezroczysta, osadza się ją jedynie w pliku HTML i przekazuje jej formularz do wyświetlenia.

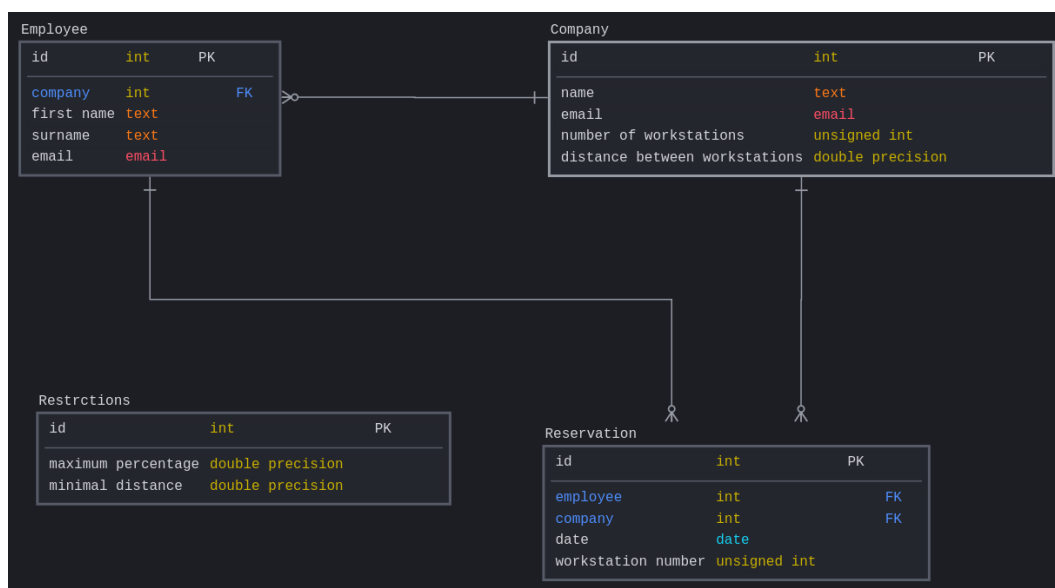
Bootstrap

Aplikacja od strony graficznej realizowana była przy pomocy biblioteki Bootstrap[4]. Jest to biblioteka języka CSS ułatwiająca tworzenie graficznego interfejsu stron internetowych, powszechnie wykorzystywana przy ich projektowaniu. Wybór ten był podyktowany łatwością instalacji i użycia tej biblioteki, jak również jej bardzo dobrą dokumentacją, opartą o wiele przykładów zastosowań. Umożliwiło to skupienie się na części backendowej aplikacji, jej poprawnym działaniu, jednocześnie nie zaniedbując szaty graficznej i ogólnego wrażenia sprawianego przez program. Dzięki wykorzystaniu tej biblioteki aplikacja wygląda przejrzysto i nowocześnie.

Heroku

Aby aplikacja była dostępna nie tylko lokalnie na komputerze, ale również na dowolnym urządzeniu podłączonym do sieci, zdecydowałem się wykorzystać platformę Heroku[6]. Umożliwia ona hostowanie aplikacji webowych, napisanych we wszystkich najpopularniejszych językach programowania. Wymaga założenia konta, ale można korzystać z niej w wersji bez żadnych opłat. Deploy aplikacji na tę platformę jest prostym procesem.

3.2. Schemat bazy danych



Rysunek 3.1: Schemat bazy danych

Uwagi do schematu:

- Wszystkie klucze główne są generowane automatycznie przez Django[2].
- Na tabelę *Reservation* jest nałożone wiązanie integralności — nie mogą pojawić się dwie krotki z tymi samymi wartościami atrybutów *Company*, *date* i *workstation number*.
- Tabele *Employee* i *Company* mają dodatkowy, niepokazany na schemacie atrybut *user*, który jest połączony w relacji *jeden-do-jeden* z systemowym obiektem *User* — jest to wymagane, by instancje tej klasy były traktowane jak użytkownicy, np. mieli hasło i mogli się nim logować.
- Tabela *Restriction* ma tylko jedną instancję z bieżącymi restrykcjami.

3.3. Opis struktury projektu

Struktura głównego katalogu

Szkielet struktury projektu jest generowany automatycznie przez framework Django[2] przy tworzeniu nowego projektu. W głównym katalogu znajduje się plik `manage.py`, będący skryptem Pythona[1], który umożliwia wykonywanie takich operacji, jak: uruchomienie serwera, przejście do shella w celu ręcznej modyfikacji bazy danych, uruchomienie testów, zmiana hasła administratora itp. Jeśli przy wyborze bazy danych zdecydowaliśmy się na korzystanie z `sqlite`, co jest opcją domyślną i zarazem wymagającą najmniej ręcznej konfiguracji, również w tym katalogu znajduje się plik `db.sqlite3`, przechowujący bazę danych. W głównym katalogu projektu znajduje się podkatalog o takiej samej nazwie jak projekt, zawierający plik `settings.py`, który jest głównym plikiem konfiguracyjnym projektu (ustawia się w nim opcje bazy danych, język, opcje walidacji haseł i wiele innych) oraz plik `urls.py`, który przechowuje mapowanie adresów url na funkcje je obsługujące. Oprócz tego, główny katalog zawiera podkatalogi aplikacji zawartych w projekcie. W terminologii Django[2] aplikacja to wydzielona część projektu, realizująca jakąś funkcjonalność. W naszym projekcie są dwie aplikacje — *users* obsługująca użytkowników (rejestrację, logowanie etc.) i *reservations* obsługująca rezerwacje (tworzenie, przeglądanie).

Struktura podkatalogu aplikacji

Szkielet katalogu aplikacji również jest generowany automatycznie przy jej tworzeniu.

Plik `models.py` jest przeznaczony na oprogramowanie bazy danych. To w nim umieszcza się tabele (jako klasy) z wszystkimi atrybutami (jako pola klasy), definiuje więzy integralności, podcina klucze obce i ewentualnie definiuje klucze główne, jeśli nie chcemy korzystać z generowanych przez Django[2].

W pliku `forms.py` definiujemy formularze, umożliwiające np. rejestrację czy zmianę danych użytkownika. Klasy formularzy są ściśle powiązane z klasami z pliku `models.py`. W klasach można również nadpisywać metody, np. `save()`, co pozwala na to, aby zaktualizowanie danych użytkownika wywołało jakąś dodatkową akcję.

Najważniejszym z punktu widzenia logiki działania stron jest plik `views.py`. Umieszczamy w nim funkcje reagujące na zdarzenia pochodzące od użytkownika, takie jak załadowanie danej strony czy naciśnięcie przycisku. Obsługujemy w nim żądania HTTP i operujemy na bazie danych.

W obu aplikacjach utworzyłem nowy plik `functions.py`, do którego wydzieliłem pomocnicze funkcje, używane w plikach `views.py` tak, aby pozostały w nich tylko funkcje odpowiedzialne za wyświetlanie poszczególnych stron. Takie rozwiązanie poprawia przejrzystość i logiczny podział kodu.

Django[2] automatycznie generuje plik `tests.py`, w którym umieszcza się testy. Można podzielić testy na mniejsze pliki, odpowiadające za poszczególne części programu. Warunkiem jest to, aby ich nazwy zaczynały się od „tests”. W mojej aplikacji wydzieliłem dwa pliki z testami: `tests_models.py` do testowania tabel bazy danych, oraz plik `tests_views.py` do testowania właściwego działania aplikacji, tj. działania poszczególnych jej stron internetowych.

Plik `admin.py` definiuje, które tabele z baz danych będą widoczne w panelu administracyjnym. Umożliwia to zarządzanie tymi tabelami przez administratora, korzystając z interfejsu dostarczanego przez Django[2].

W podkatalogu aplikacji może też znajdować się plik `urls.py`, rozszerzający mapowanie stron na wyświetlające je funkcje, znajdujące się w pliku o tej samej nazwie w głównym katalogu projektu.

W podkatalogu `templates` znajdują się pliki HTML zawierające szablony wyświetlanych przez użytkownika stron. Szablony mogą po sobie dziedziczyć — dzięki temu mamy jeden szablon na ogólną szatę graficzną naszej aplikacji, i podszablony do poszczególnych widoków. Zawierają one osadzone fragmenty kodu Pythona[1], który umożliwia zarządzanie wyświetlaniem zmiennych przekazywanych do tych szablonów.

3.4. Testowanie

Jak napisano w poprzednim podrozdziale, testy są podzielone na dwie części: testy obsługi bazy danych, znajdujące się w plikach `tests_models.py`, i testy funkcjonalne, sprawdzające poprawność operacji wykonywanych przez użytkowników, które znajdują się w plikach `tests_views.py`. Aby uruchomić testy, należy otworzyć terminal w katalogu głównym projektu i wykonać polecenie

```
$ python manage.py test
```

Komenda ta uruchomi wszystkie testy w projekcie. Można wyszczególnić, które testy chce się uruchomić, przez podanie aplikacji, pliku, klasy i konkretnego testu, np. polecenie

```
$ python manage.py test users.tests_views.UpdatePasswordTests
```

uruchomi wszystkie testy z klasy `UpdatePasswordTests`, znajdujące się w pliku `tests_views.py` w aplikacji `users`. Poniżej zamieszczone są dwa przykłady testów — jeden sprawdzający obsługę bazy danych, i drugi sprawdzający poprawne działanie widoku.

Pierwszy test pochodzi z pliku `tests_model.py` z aplikacji `users`.

```
def test_date_validation(self):
    reservation = Reservation(
        user=self.profile, company=self.company, date='10/02/2022',
        workstation_number=1)
    with self.assertRaises(ValidationError):
        reservation.save()
```

Powyższy test sprawdza, czy przy próbie zapisu do bazy danych rezerwacji z datą w niepoprawnym formacie (poprawny to YYYY-mm-dd, np. 2022-02-10) nastąpi błąd walidacji.

Drugi test pochodzi z pliku `tests_views.py` z aplikacji `reservations`.

```
def test_employee_user(self):
    is_logged = self.client.login(
        username='test2', password='testing123')
    self.assertEqual(is_logged, True)
    response = self.client.get('/history-of-reservations/')
    self.assertEqual(response.context['user_type'], 'profile')
    self.assertEqual(response.status_code, 200)
    self.assertTemplateUsed(response, 'reservations/history.html')
    self.assertEqual(response.context['page_obj'].number, 1)
    self.assertEqual(response.context['page_obj'].paginator.num_pages,
        2)
    self.assertEqual(response.context['page_obj'].paginator.count, 20)
    self.assertEqual(len(response.context['page_obj']), 14)
```

Ten test pochodzi z klasy `HistoryTests` i ma za zadanie sprawdzić, czy dla użytkownika typu pracownik poprawnie wyświetla się historia rezerwacji. Asercje sprawdzają status odpowiedzi HTTP, czy załadowana została poprawna strona html, czy zgadza się typ użytkownika, oraz paginację: czy jesteśmy na pierwszej stronie z dwóch, czy liczba rezerwacji wynosi 20 i czy na bieżącej stronie wyświetlanych jest 14 z nich.

Rozdział 4.

Zakończenie

4.1. Podsumowanie

Efektem tej pracy jest aplikacja, która istotnie ułatwia zarówno pracodawcom jak i pracownikom organizację pracy stacjonarnej podczas pandemii. Jako aplikacja internetowa nie wymaga instalacji ani posiadania na komputerze żadnego dodatkowego oprogramowania. Jest prosta w obsłudze, jej interfejs jest czytelny, łatwy do opanowania i nowoczesny. Program wybiera dostępne miejsca w oparciu o dwa kryteria. Jest dostosowany do zmiany restrykcji w czasie. Umożliwia zarówno tworzenie rezerwacji, jak i przegląd już dokonanych z wyróżnieniem tych, dotyczących przyszłych dat. Aplikacja jest zatem kompletna i spełnia wszystkie założone funkcjonalności.

Aplikacja może być używana we wszystkich miejscach pracy biurowej, które nie przeszły w całości na tryb zdalny. Nie ma tu znaczenia, czy jest to prywatna firma, czy instytucja publiczna, jak np. urząd czy uczelnia. Grono potencjalnych odbiorców jest zatem ogromne.

4.2. Możliwe kierunki rozwoju

Napisana aplikacja jest kompletna, jednak zawsze można rozszerzyć ją o pewne dodatkowe funkcjonalności. Poniżej znajduje się kilka z nich, które w moim odczuciu byłyby najbardziej oczekiwane przez użytkowników:

Zmiana graficznego sposobu wyświetlania dostępnych miejsc przy rezerwacji. Zamiast listy numerów stanowisk i przycisków „Book” pojawiałby się plan biura, pracownik zaznaczałby miejsce na planie. Taki sposób prezentacji byłby bardziej przejrzysty.

Zmiana struktury bazy danych tak, aby pracownik mógł wskazać więcej niż jedną firmę. Obecnie osoba pracująca w kilku firmach jednocześnie musi założyć w aplikacji kilka kont, (imię, nazwisko, email i hasło mogą być identyczne, ale *username* musi być różny) po jednym dla każdego miejsca zatrudnienia. Po takiej zmianie tacy pracownicy mogliby mieć tylko jedno konto, z którego obsługiwaliby wszystkie swoje prace.

Wysyłanie powiadomień przez email. Na adres podany przy zakładaniu konta byłyby wysyłane powiadomienia np. o zbliżającej się rezerwacji, czy zmianie restrykcji i w związku z tym anulowaniu przyszłych rezerwacji.

Bibliografia

- [1] Dokumentacja języka Python, <https://docs.python.org/3/>
- [2] Dokumentacja frameworka Django, <https://docs.djangoproject.com/en/4.0/>
- [3] Writing your first Django app, <https://docs.djangoproject.com/en/4.0/intro/>
- [4] Dokumentacja biblioteki Bootstrap, <https://getbootstrap.com/docs/5.1/getting-started/introduction/>
- [5] Dokumentacja biblioteki NumPy, <https://numpy.org/doc/>
- [6] Dokumentacja Heroku dla Django, <https://devcenter.heroku.com/categories/working-with-django>
- [7] Dokumentacja Django-crispy-forms, <https://django-crispy-forms.readthedocs.io/en/latest/>
- [8] Rozporządzenie Rady Ministrów z dnia 10 kwietnia 2020 r. w sprawie ustanowienia określonych ograniczeń, nakazów i zakazów w związku z wystąpieniem stanu epidemii, <https://isap.sejm.gov.pl/isap.nsf/download.xsp/WDU20200000658/0/D20200658.pdf>
- [9] Rozporządzenie Rady Ministrów z dnia 19 czerwca 2020 r. w sprawie ustanowienia określonych ograniczeń, nakazów i zakazów w związku z wystąpieniem stanu epidemii, <https://isap.sejm.gov.pl/isap.nsf/download.xsp/WDU20200001066/0/D20201066.pdf>
- [10] Rozporządzenie Rady Ministrów z dnia 7 sierpnia 2020 r. w sprawie ustanowienia określonych ograniczeń, nakazów i zakazów w związku z wystąpieniem stanu epidemii, <https://isap.sejm.gov.pl/isap.nsf/download.xsp/WDU20200001356/0/D20201356.pdf>
- [11] Jeff Forcier, Paul Bissex, Wesley Chun, *Python i Django. Programowanie aplikacji webowych*, Helion, 2009